



Name: Sharon Elango

Batch : 3 Roll No : 62

Expt.-8: Clustering of Countries with World Bank API

```
In [64]: !pip install requests
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests) (3.4.5)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (2026.2.25)
```

```
In [65]: import requests
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler
from scipy.cluster.hierarchy import dendrogram, linkage
```

```
In [66]: def fetch_indicator(code, year = "2022"):
    url = f"https://api.worldbank.org/v2/country/all/indicator/{code}?date={year}"
    response = requests.get(url).json()[1]

    data = {}
    for item in response:
        country = item["country"]["value"]
        value=item["value"]
        if value is not None:
            data[country] = value

    return pd.Series(data, name=code)
```

```
In [67]: gdp = fetch_indicator("NY.GDP.PCAP.CD")
pop = fetch_indicator("SP.POP.TOTL")
life = fetch_indicator("SP.DYN.LE00.IN")
lit = fetch_indicator("SE.ADT.LITR.ZS")
```

```
In [68]: df=pd.concat ([gdp, pop, life, lit], axis=1)
```

```
df.columns=["GDP", "Population", "LifeExpectancy", "Literacy"]
df.dropna(inplace=True)
df.head()
```

Out[68]:

	GDP	Population	LifeExpectancy	Literacy
Africa Eastern and Southern	1679.327622	731821393	64.487020	73.055977
Africa Western and Central	2138.473153	497387180	57.987813	60.780979
Arab World	7950.355820	471352066	71.876096	78.271927
Central Europe and the Baltics	19530.155323	100071871	76.652299	99.479919
Early-demographic dividend	4043.827763	3470644834	71.430455	81.242798

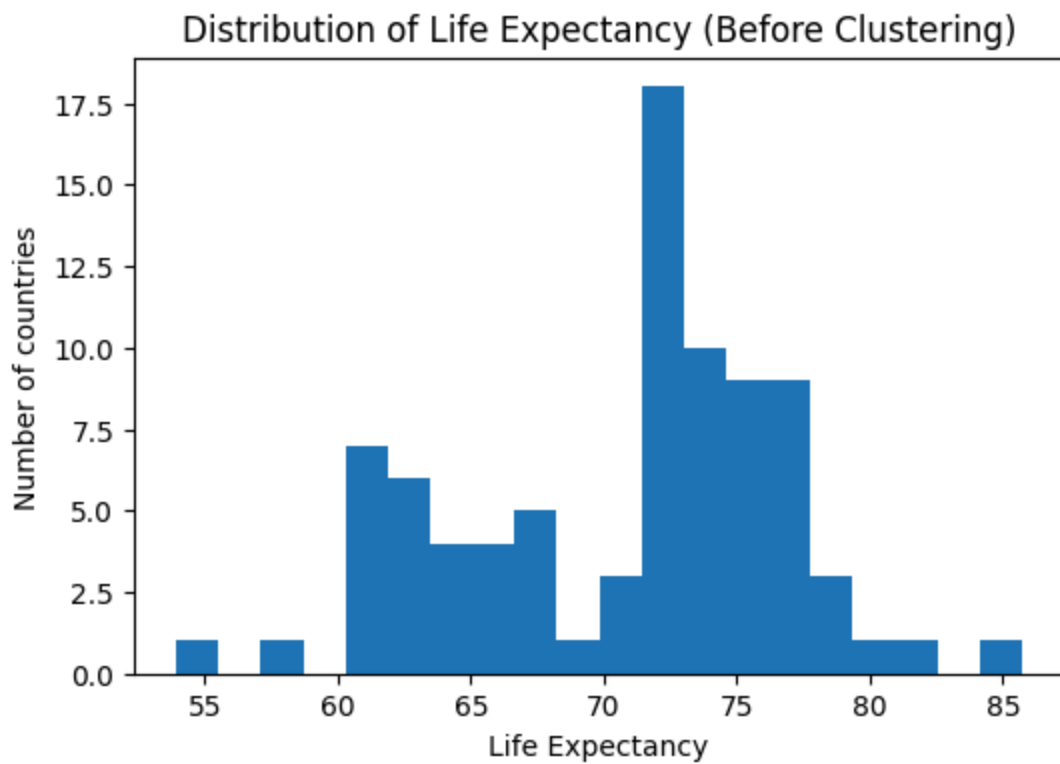
In [69]: df.tail()

Out[69]:

	GDP	Population	LifeExpectancy	Literacy
Uruguay	20818.614609	3390913	76.468	98.849998
Uzbekistan	2578.666894	34938955	72.147	100.000000
Viet Nam	4147.697772	99680655	74.502	96.129997
West Bank and Gaza	3799.955270	5043612	76.662	97.839996
Zambia	1447.123101	20152938	65.279	79.975143

In [70]: scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

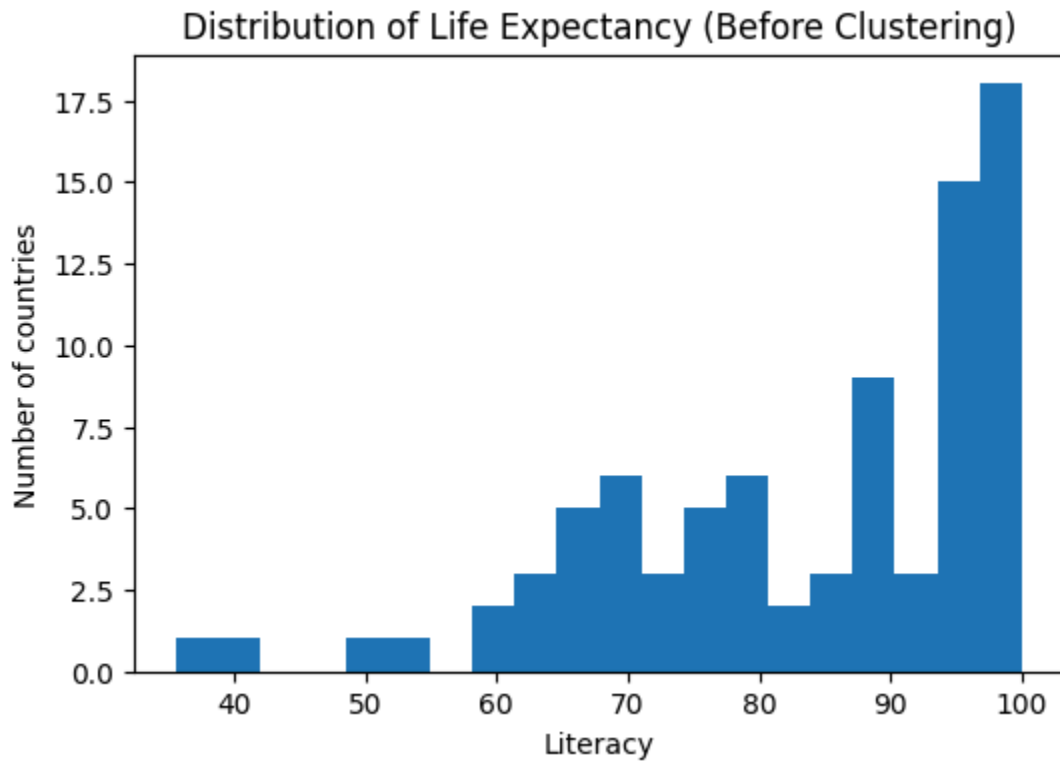
In [71]: plt.figure(figsize=(6,4))
plt.hist(df["LifeExpectancy"], bins = 20)
plt.xlabel("Life Expectancy")
plt.ylabel("Number of countries")
plt.title("Distribution of Life Expectancy (Before Clustering)")
plt.show()



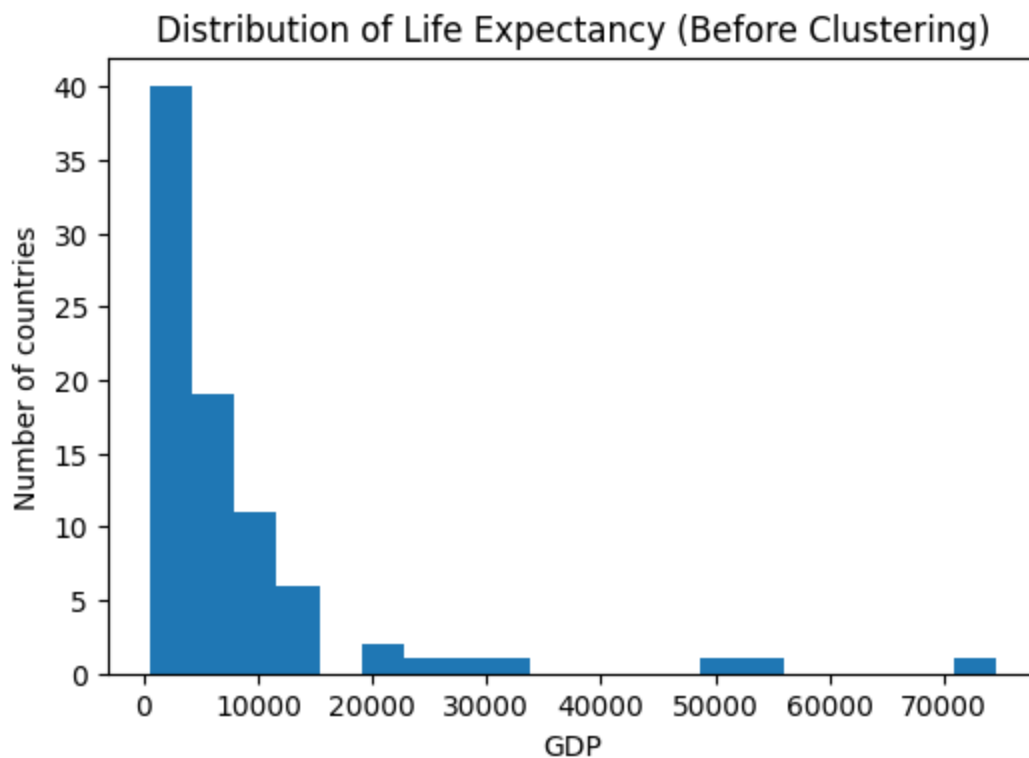
```
In [72]: plt.figure(figsize=(6,4))
plt.hist(df["Population"], bins = 20)
plt.xlabel("Population")
plt.ylabel("Number of countries")
plt.title("Distribution of Life Expectancy (Before Clustering)")
plt.show()
```



```
In [73]: plt.figure(figsize=(6,4))
plt.hist(df["Literacy"], bins = 20)
plt.xlabel("Literacy")
plt.ylabel("Number of countries")
plt.title("Distribution of Life Expectancy (Before Clustering)")
plt.show()
```



```
In [74]: plt.figure(figsize=(6,4))
plt.hist(df["GDP"], bins = 20)
plt.xlabel("GDP")
plt.ylabel("Number of countries")
plt.title("Distribution of Life Expectancy (Before Clustering)")
plt.show()
```



```
In [75]: country_name = "India"

result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])
```

```
Country: India
GDP per capita: 2347.44829434623
Population: 1425423212.0
Life Expectancy: 71.698
Literacy Rate: 76.3199996948242
```

```
In [76]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
```

```

scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc

# Now, retrieve and print information for 'India'
country_name = "India"

# Check if India is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly

```

```

Country: India
GDP per capita: 2347.44829434623
Population: 1425423212.0
Life Expectancy: 71.698
Literacy Rate: 76.3199996948242
KMeans Cluster: 0.0
Hierarchical Cluster: 1.0

```

```

In [77]: country_name = "Brazil"

result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])

```

Country: Brazil
GDP per capita: 9281.33282136864
Population: 210306415.0
Life Expectancy: 74.872
Literacy Rate: 94.3853874531348

```
In [78]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life)
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc

# Now, retrieve and print information for 'Brazil'
country_name = "Brazil"

# Check if Brazil is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly
```

Country: Brazil
GDP per capita: 9281.33282136864
Population: 210306415.0
Life Expectancy: 74.872
Literacy Rate: 94.3853874531348
KMeans Cluster: 1.0
Hierarchical Cluster: 0.0

```
In [79]: country_name = "North America"

result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])
```

Country: North America
GDP per capita: 74535.6365392629
Population: 373018004.0
Life Expectancy: 77.8327679594239
Literacy Rate: 98.7891464233398

```
In [80]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc
```



```

# Now, retrieve and print information for 'North America'
country_name = "North America"

# Check if North America is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly

```

Country: North America
GDP per capita: 74535.6365392629
Population: 373018004.0
Life Expectancy: 77.8327679594239
Literacy Rate: 98.7891464233398
KMeans Cluster: 1.0
Hierarchical Cluster: 0.0

```

In [81]: country_name = "Viet Nam"

result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])

```

Country: Viet Nam
GDP per capita: 4147.69777213621
Population: 99680655.0
Life Expectancy: 74.502
Literacy Rate: 96.129997253418

```

In [82]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

```

```

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc

# Now, retrieve and print information for 'Viet Nam'
country_name = "Viet Nam"

# Check if Viet Nam is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly

```

```

Country: Viet Nam
GDP per capita: 4147.69777213621
Population: 99680655.0
Life Expectancy: 74.502
Literacy Rate: 96.129997253418
KMeans Cluster: 1.0
Hierarchical Cluster: 0.0

```

```

In [83]: country_name = "Bahrain"

result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])

```

```
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])
```

Country: Bahrain
GDP per capita: 30470.5219276023
Population: 1524693.0
Life Expectancy: 80.992
Literacy Rate: 97.870002746582

```
In [84]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc

# Now, retrieve and print information for 'Bahrain'
country_name = "Bahrain"

# Check if Bahrain is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly
```

Country: Bahrain
GDP per capita: 30470.5219276023
Population: 1524693.0
Life Expectancy: 80.992
Literacy Rate: 97.870002746582
KMeans Cluster: 1.0
Hierarchical Cluster: 0.0

```
In [85]: #Consequences of removing dropna :
import pandas as pd
# After removing dropna :
df=pd.concat([gdp, pop, life, lit], axis=1)
df.columns=["GDP", "Population", "LifeExpectancy", "Literacy"]
#df.dropna(inplace=True)
df.head()
```

```
Out[85]:
```

	GDP	Population	LifeExpectancy	Literacy
Africa Eastern and Southern	1679.327622	731821393	64.487020	73.055977
Africa Western and Central	2138.473153	497387180	57.987813	60.780979
Arab World	7950.355820	471352066	71.876096	78.271927
Caribbean small states	17456.308485	4497310	72.832292	NaN
Central Europe and the Baltics	19530.155323	100071871	76.652299	99.479919

```
In [86]: country_name = "United States"
result = df.loc[country_name]

print("Country:", country_name)
print("GDP per capita:", result["GDP"])
print("Population:", result["Population"])
print("Life Expectancy:", result["LifeExpectancy"])
print("Literacy Rate:", result["Literacy"])
#print("KMeans Cluster:", result["kMeansCluster"])
#print("Hierarchical Cluster:", result["Hierarchical Cluster"])
```

Country: United States
GDP per capita: 76657.2488844403
Population: 334017321.0
Life Expectancy: 77.4341463414634
Literacy Rate: nan

```
In [87]: import pandas as pd
from sklearn.cluster import KMeans, AgglomerativeClustering
from sklearn.preprocessing import StandardScaler

# 1. Re-create and CLEAN the dataframe (from previously defined gdp, pop, life
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]
```

```

# CRITICAL: drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# 3. K-Means Clustering (using k=3 as determined by elbow method earlier)
km = KMeans(n_clusters=3, random_state=0, n_init=10) # n_init avoids warnings
labels_km = km.fit_predict(X_scaled)

# Add KMeans labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km

# 4. Hierarchical Clustering (using n_clusters=3)
hc = AgglomerativeClustering(n_clusters=3)
labels_hc = hc.fit_predict(X_scaled)

# Add Hierarchical labels back to the cleaned dataframe
df["Hierarchical Cluster"] = labels_hc

# Now, retrieve and print information for 'United States'
country_name = "United States"

# Check if United States is in the dataframe after dropping NaNs
if country_name in df.index:
    result = df.loc[country_name]

    print("Country:", country_name)
    print("GDP per capita:", result["GDP"])
    print("Population:", result["Population"])
    print("Life Expectancy:", result["LifeExpectancy"])
    print("Literacy Rate:", result["Literacy"])
    print("KMeans Cluster:", result["KMeansCluster"])
    print("Hierarchical Cluster:", result["Hierarchical Cluster"])
else:
    print(f"'{country_name}' not found in the dataset after cleaning (possibly

```

'United States' not found in the dataset after cleaning (possibly due to missing data).

```

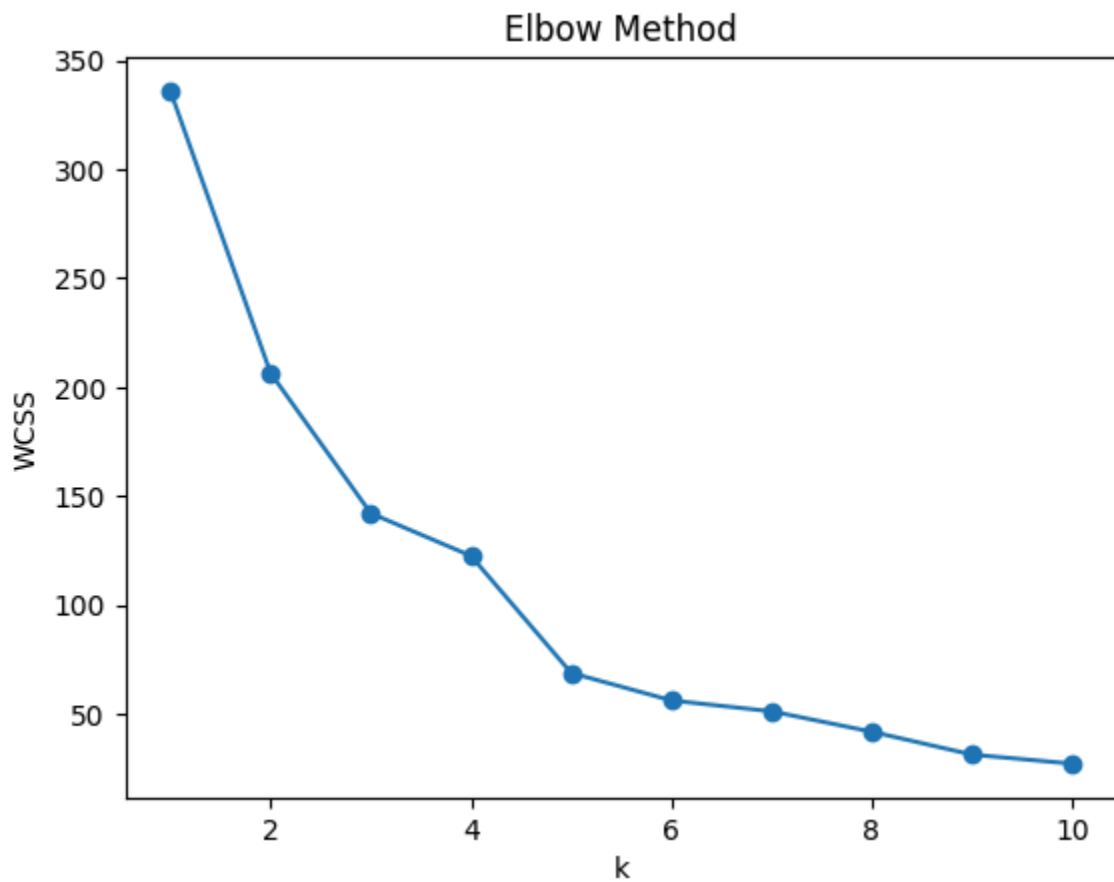
In [88]: #Elbow Method to find optimal K
#This loop tests k values
WCSS = []
for k in range(1,11):
    km = KMeans(n_clusters=k, random_state=0)
    km.fit(X_scaled)
    WCSS.append(km.inertia_)

print("k\tWCSS")
for i in range(10):
    print(i+1, "\t", WCSS[i])

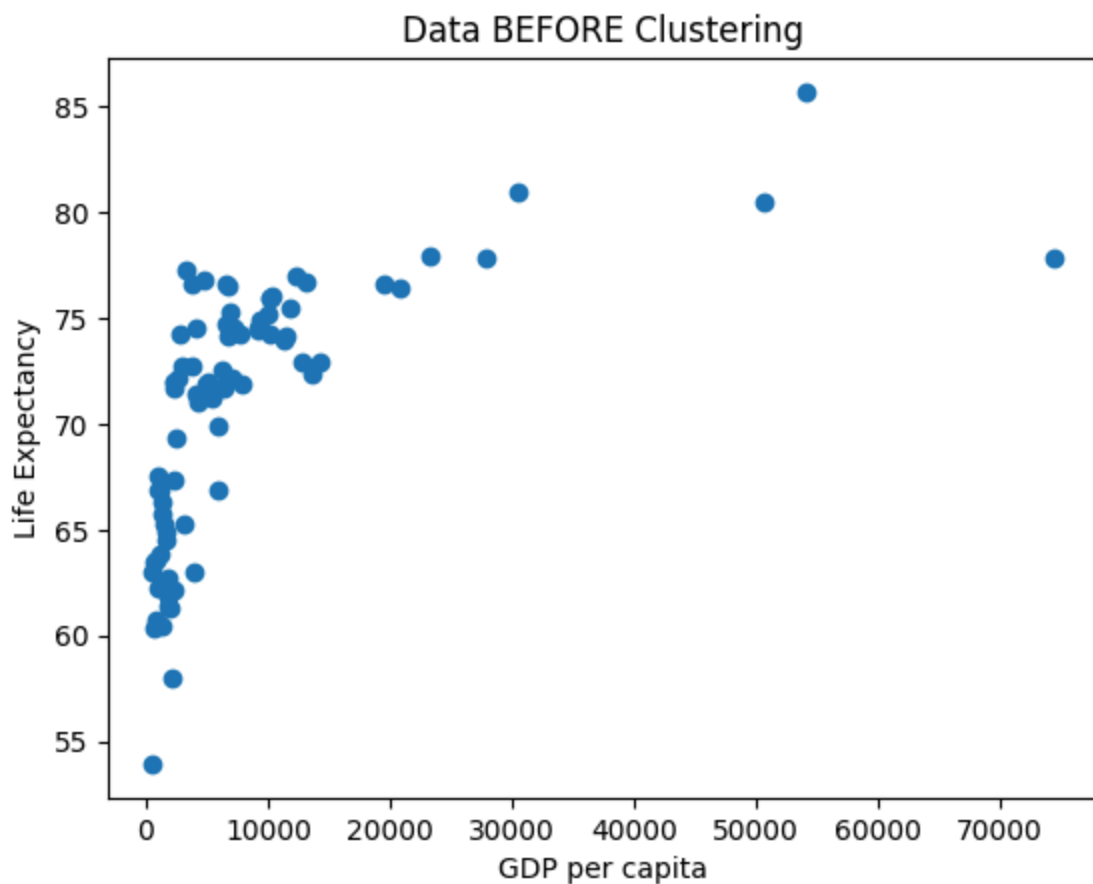
```

```
plt.plot(range(1,11), WCSS, marker='o')
plt.xlabel("k")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()
```

k	WCSS
1	336.00000000000006
2	206.43717192603808
3	142.06028324967596
4	122.46400641421171
5	68.71234707726028
6	56.092718405819106
7	51.08265517326418
8	41.70851027024395
9	31.213863029560223
10	27.04725098650771



```
In [89]: #Part-3: Clustering
plt.scatter(df["GDP"], df["LifeExpectancy"])
plt.xlabel("GDP per capita")
plt.ylabel("Life Expectancy")
plt.title("Data BEFORE Clustering")
plt.show()
```



```
In [90]: # 1. Re-create and CLEAN the dataframe
df = pd.concat([gdp, pop, life, lit], axis=1)
df.columns = ["GDP", "Population", "LifeExpectancy", "Literacy"]

# CRITICAL: You MUST drop NaNs for K-Means to work
df.dropna(inplace=True)

# 2. Scale the data AFTER cleaning
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

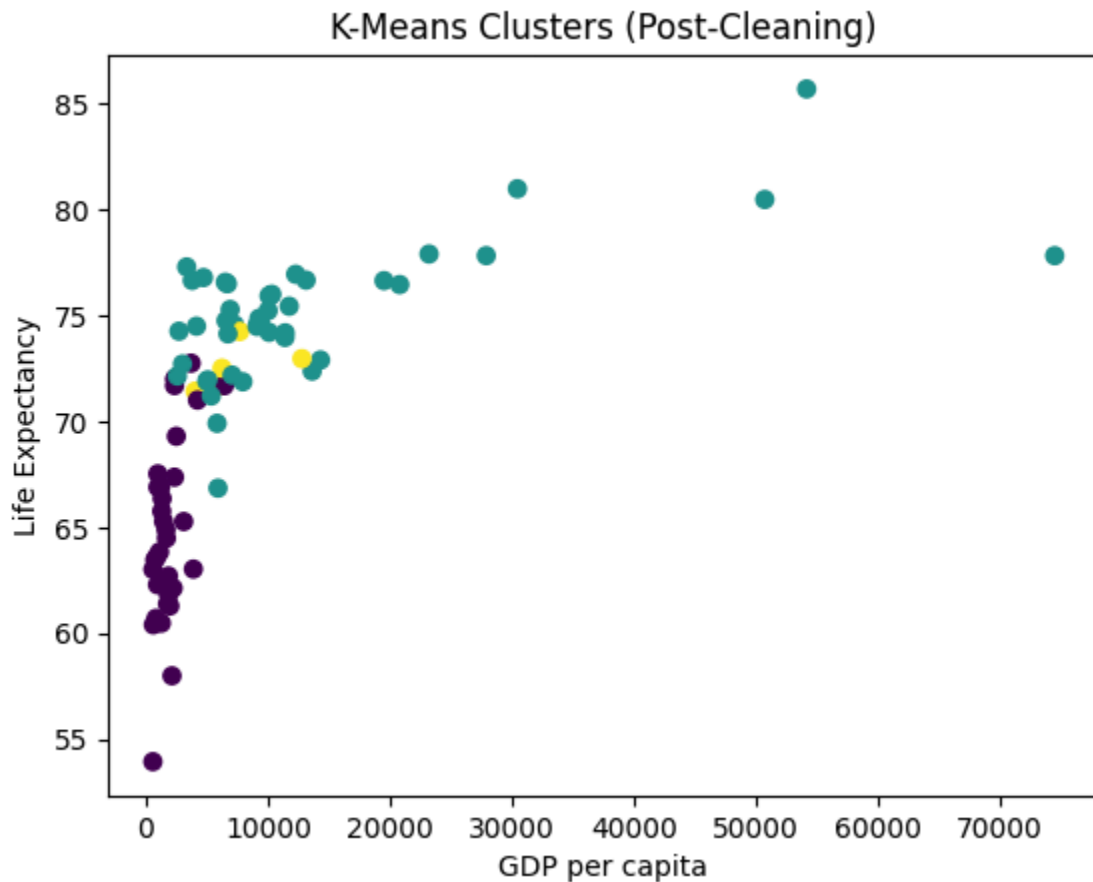
# 3. Elbow Method
WCSS = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=0, n_init=10) # n_init avoids warni
    km.fit(X_scaled)
    WCSS.append(km.inertia_)

# ... (Your plotting code for Elbow Method) ...

# 4. K-Means Clustering
km = KMeans(n_clusters=3, random_state=0, n_init=10)
labels_km = km.fit_predict(X_scaled)

# Add labels back to the cleaned dataframe
df["KMeansCluster"] = labels_km
```

```
# 5. Visualization
plt.scatter(df["GDP"], df["LifeExpectancy"], c=df["KMeansCluster"], cmap='viridis')
plt.xlabel("GDP per capita")
plt.ylabel("Life Expectancy")
plt.title("K-Means Clusters (Post-Cleaning)")
plt.show()
```



```
In [91]: cluster_summary = df.groupby("KMeansCluster")[["GDP", "Population", "LifeExpectancy", "Literacy"]]
print(cluster_summary)
```

KMeansCluster	GDP	Population	LifeExpectancy	Literacy
0	1915.596661	6.544831e+08	64.856833	68.564031
1	13280.490845	3.923248e+08	75.101970	94.739797
2	7094.638897	5.911254e+09	72.421619	86.116721

```
In [92]: hc = AgglomerativeClustering(n_clusters = 3)
labels_hc = hc.fit_predict(X_scaled)
df["HierarchicalCluster"] = labels_hc
plt.scatter(df["GDP"], df["LifeExpectancy"], c = labels_hc)
plt.xlabel("GDP per capita")
plt.ylabel("Life Expectancy")
plt.title("Hierarchical clustering of countries")
plt.show()
```


Hierarchial clustering of countries

